

ReBook

Book Exchange App

Final Project – Software Engineering

Ayesha Jan

Maha Ibrahim

Anunitha KU

Masa Awaineh

Sinchana Basavaraj

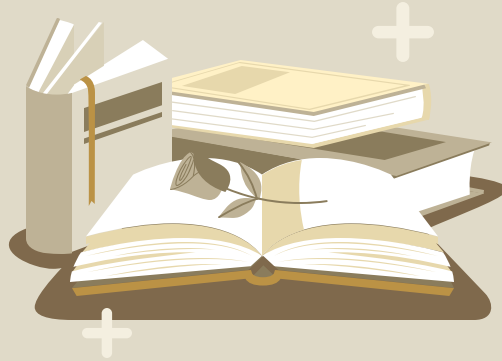


Table of contents



01.

SDLC Model



02.

UML Diagrams



03.

Design and
Architectural
Aspects

04.

Testing
Strategy



05.

Project
Management



Introduction

Book Exchange App

- A community-based platform that allows users to trade and share used books easily.
- Enables users to register, create profiles, list books with details, and search or request books from others.
- The app supports in-app messaging for communication, notifications for updates on exchanges, and an admin panel for managing users, books, and categories.
- Overall, it promotes sustainability, encourages reading, and connects book lovers within their local community.



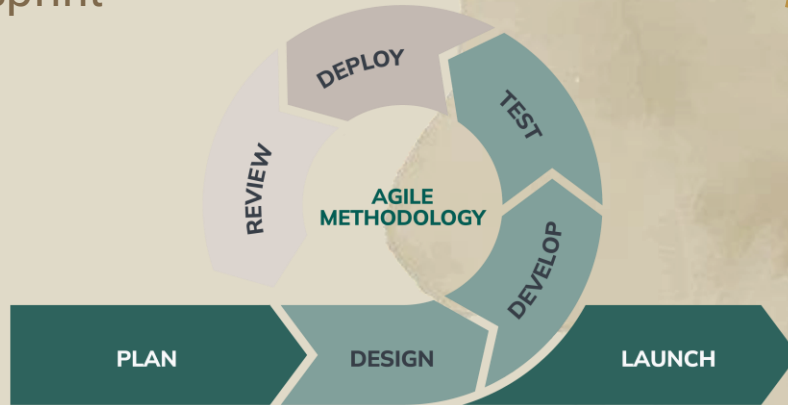
01

SDLC Model



Why We Chose the Agile Model?

- Continuous feedback and flexibility
- Fast and iterative delivery
- Strong collaboration and adaptability
- Testing and improvement in every sprint



Why Agile Fits Our Project

Feature updates and new ideas

Iterative sprints allow
adding features gradually

Frequent user feedback

Agile welcomes changes anytime

Close teamwork

Scrum promotes daily
collaboration

Early testing

Each sprint includes testing

Quick MVP release

Focuses on delivering
working software early

Agile vs. Other SDLC Models

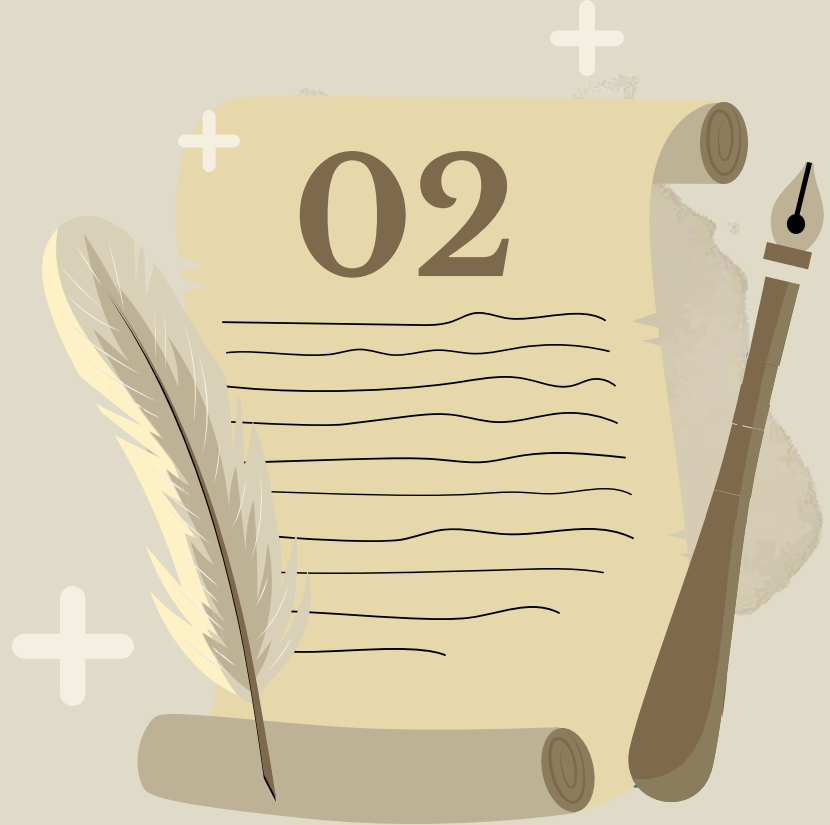
Waterfall	Too rigid, no room for late changes
V-Model	Testing-heavy, not flexible for evolving features
Spiral	Overly complex and time-consuming for our scope
RAD	Fast but lacks ongoing feedback and adaptability
Iterative Incremental	Structured but less team-driven than Agile.
Prototyping	Focused on early demos, not full product evolution
Agile (Chosen)	Flexible, collaborative, iterative, user-focused

Application in ReBook

	Goal	Duration
Sprint 1	Week 1–2	User registration & login
Sprint 2	Week 3–4	Book listing & search
Sprint 3	Week 5–6	Exchange system & chat
Sprint 4	Week 7-8	Admin Features
Sprint 5	Week 9-10	Testing & final fixes

UML Diagrams

Behavioral and
Structural Diagrams



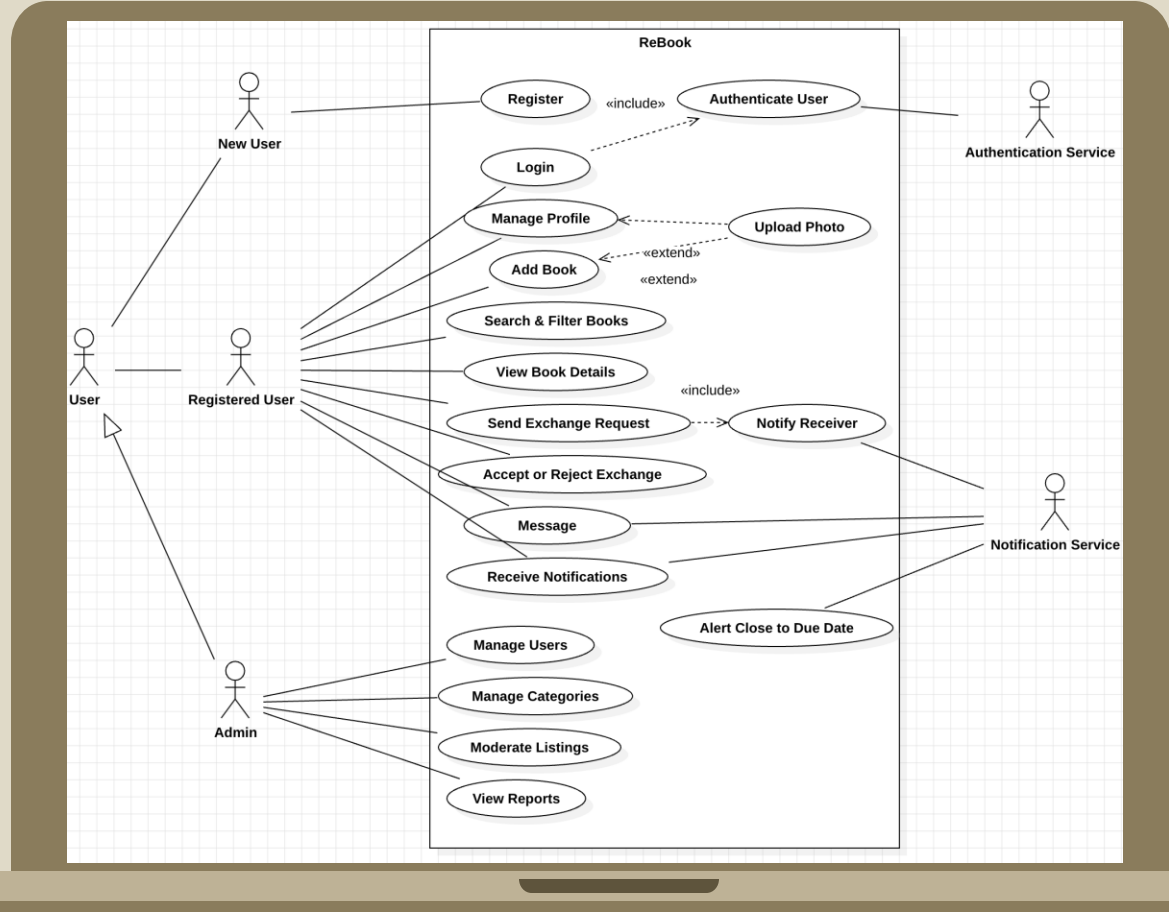


02-01

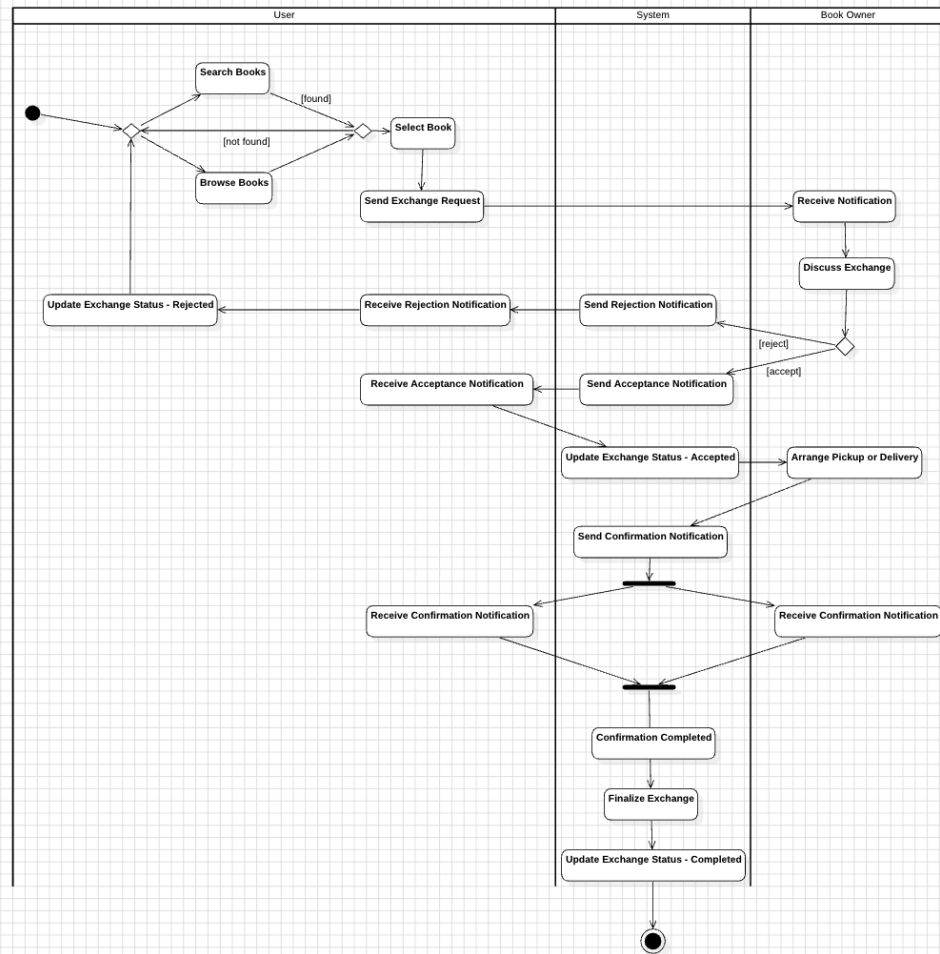
Behavioral Diagrams

Use Case Diagram

- Users register, log in, add books, and request exchanges
- Admins manage users, categories, and listings
- External services handle authentication and notifications



Activity Diagram



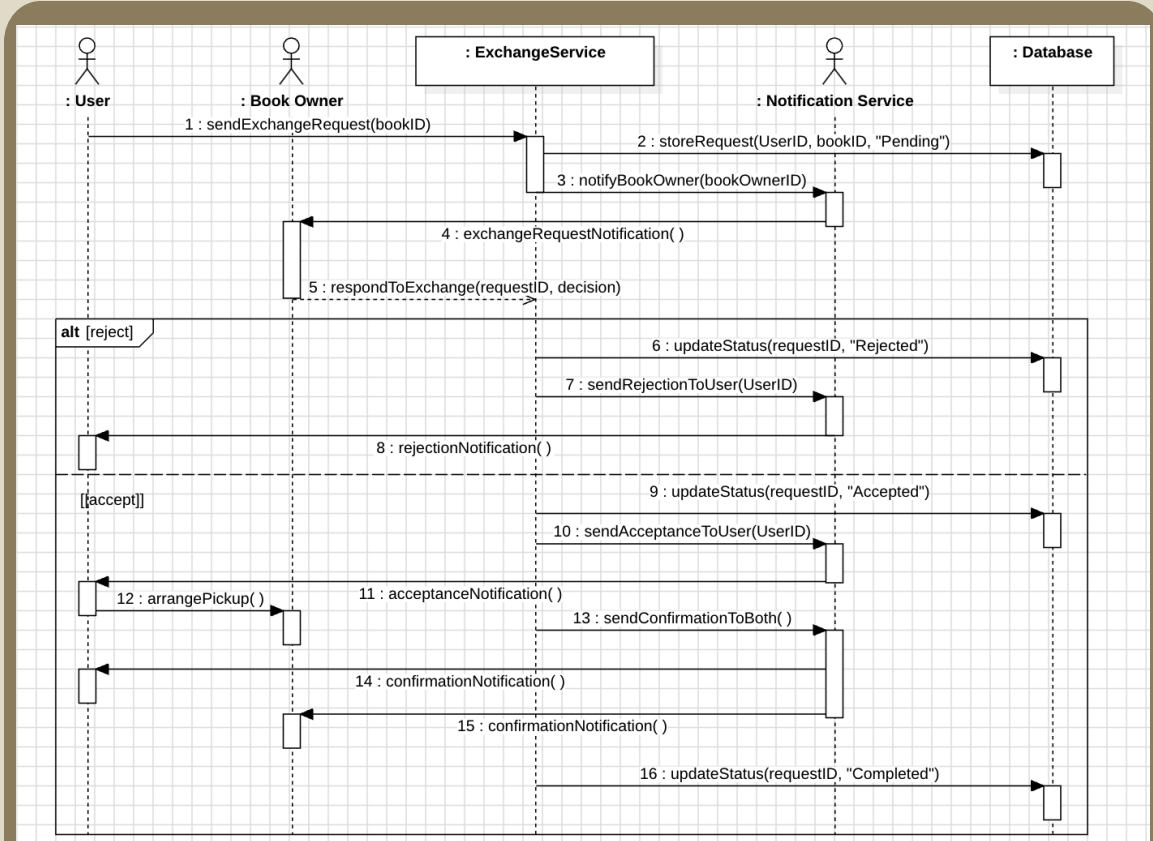
- 3 Swimlanes: User | System | Book Owner
- User searches/browses for a book
- Sends exchange request to the owner
- System notifies the owner
- Owner accepts or rejects
- Reject: user notified, status updated
- Accept: both confirm, exchange completed
- System finalizes and updates status

Sequence Diagram

- **Lifelines:** User, Book Owner, ExchangeService, Notification Service, Database.
- User sends exchange request → system stores it (Pending) and notifies the owner.

Owner accepts or rejects:

- Reject: status → Rejected, user notified.
- Accept: status → Accepted, pickup arranged, confirmation sent.
- System updates to Completed





02-02

Structural Diagrams

Key Classes: User, Book, Exchange, Message, Notification, Admin.

- ```

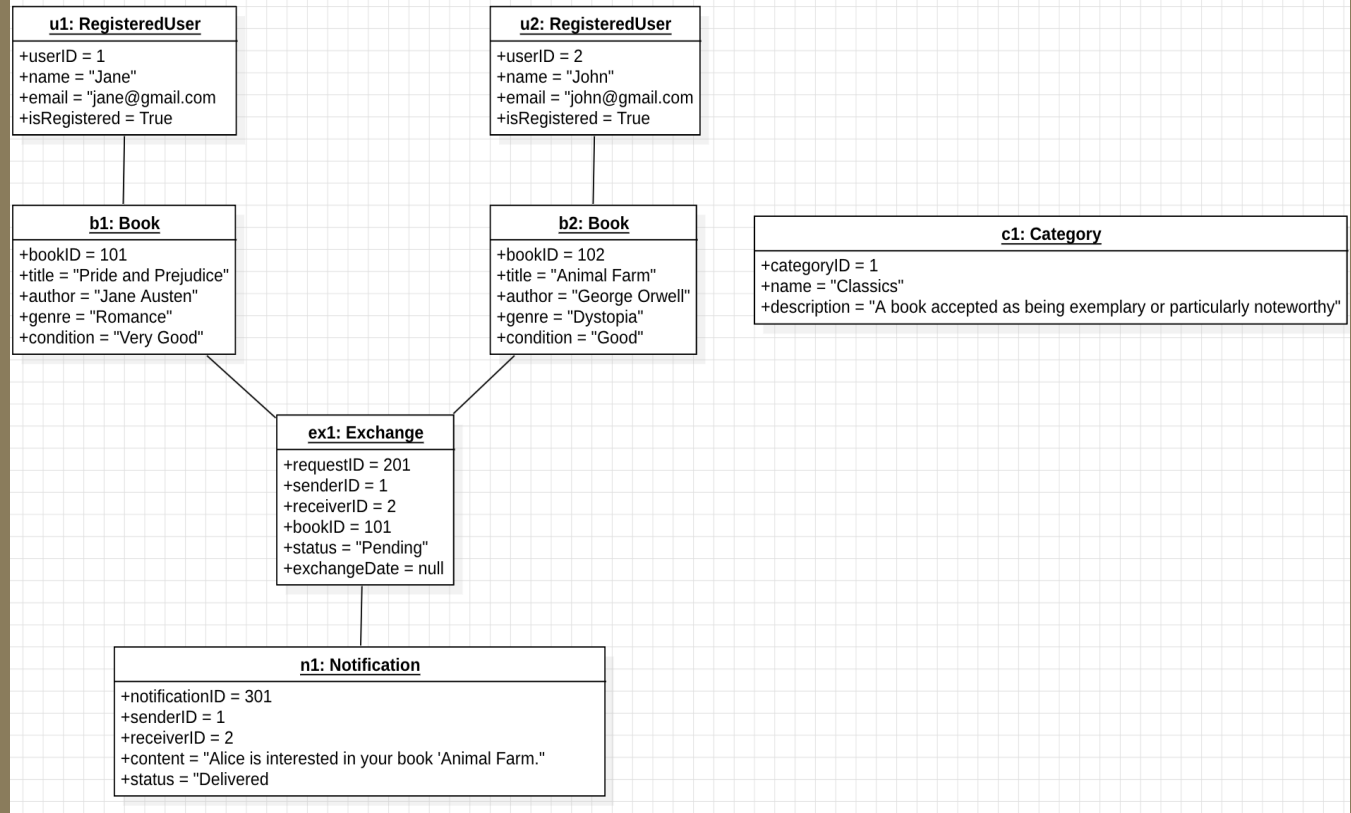
classDiagram
 class User {
 -userID: int
 -name: String
 -email: String
 -password: String
 -isRegistered: boolean
 +login()
 +updateProfile()
 +sendMessage()
 +receiveMessage()
 +sendExchangeRequest()
 +receiveExchangeRequest()
 +receiveNotification()
 }
 class NewUser {
 +register()
 }
 class RegisteredUser {
 +addBook()
 +removeBook()
 +uploadPhoto()
 +searchBooks()
 +viewBookDetails()
 +acceptRequest()
 +rejectRequest()
 +reportUser()
 }
 class Admin {
 +manageUsers()
 +manageCategories()
 +manageBooks()
 +viewReports()
 }
 class Book {
 -bookID: int
 -title: String
 -author: String
 -genre: String
 -condition: String
 -photo: String
 }
 class Category {
 -categoryID: int
 -name: String
 -description: String
 +addCategory()
 +removeCategory()
 +updateCategory()
 }
 class Exchange {
 -requestID: int
 -senderID: int
 -receiverID: int
 -bookID: int
 -status: String
 -exchangeDate: Date
 +processSendRequest()
 +processResponse()
 +updateStatus()
 }
 class Message {
 -messageID: int
 -senderID: int
 -receiverID: int
 -content: String
 -timestamp: DateTime
 }
 class Notification {
 -notificationID: int
 -senderID: int
 -receiverID: int
 -content: String
 -type: String
 -status: String
 }
 class NotificationService {
 +notifyReceiver()
 +sendConfirmation()
 +sendRejection()
 +sendAcceptance()
 +sendReminder()
 }

 NewUser --|> User
 RegisteredUser --|> User
 Admin --|> User
 User "1" -- "0..*" Message : +sends
 User "1" -- "0..*" Message : +receives
 User "1" -- "0..*" Notification : +sends
 User "1" -- "0..*" Notification : +receives
 User "1" -- "0..*" Exchange : +sends
 User "1" -- "0..*" Exchange : +receives
 User "1" -- "0..*" Book : +owns
 Book "1" -- "0..*" Exchange : +involves
 Book "0..*" -- "1" Category : +belongsTo
 Exchange "1" -- "0..*" Notification : +generates
 NotificationService ..> Notification : +notifyReceiver()

```

# Object Diagram

- u1 & u2: Actual users (Jane and John) active in the system.
- b1 & b2: Their real listed books.
- c1: A concrete category ("Classics").
- ex1: A live exchange request from Jane to John (Pending).
- n1: A delivered notification alerting John about the request.







# How each diagram contributes to understanding ReBook



## Use Case

Shows main user interactions and system functions.

## Activity

Outlines the flow of actions in a book exchange.

## Sequence

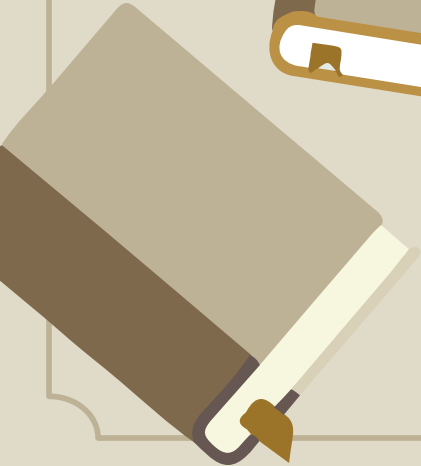
Details message order between users and system components.

## Class

Defines system structure, classes, and relationships.

## Object

Displays real-time instances and data links during an exchange.





03

# Design and Architectural Aspects

# Architectural pattern applied

- Our Book Exchange App follows a 3 Tier Architecture, dividing the system into three logical layers
  - Presentation Layer (UI layer)
  - Business Logic Layer (Exchange rules, notifications, user management)
  - Data Layer (Storage of users, books, exchanges)

This ensures separation of concerns, with each layer handling a different responsibility which makes the system modular, maintainable and scalable.

# Why it fits our use case

|                               |                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Separation of concerns</b> | Each layer focuses on a specific task: UI handles interaction, logic layer processes, data layer stores. |
| <b>Scalability</b>            | As users or listings grow, logic and database layers can scale independently.                            |
| <b>Maintainability</b>        | A change in one layer (e.g., redesigning UI) doesn't affect others.                                      |
| <b>Security</b>               | UI can't access the database directly; all operations go through APIs.                                   |
| <b>Parallel development</b>   | Frontend and backend teams can develop and test simultaneously.                                          |

# Presentation layer (UI)

- Interface through which users interact with the system.
- Flutter
- **Responsibilities:**
  - User registration and login forms.
  - Displaying books, requests, and notifications.
  - Sends HTTP requests (GET, POST) to the backend API endpoints (/books, /exchange/request, /exchange/accept, /profile)
- **Design Principle Applied:**
  - Loose Coupling* → communicates with backend only through REST APIs.
  - High Cohesion* → UI components only handle presentation, not logic.

# Business Logic Layer

- Processes all functional rules and decisions of the app.
- Python (FastAPI)
- **Responsibilities:**
  - Handles book exchange requests and messaging logic.
  - Validates user actions (e.g., checks if a book exists, if a request is valid).
  - Sends notifications to involved users.
  - Acts as a mediator between the UI and the database
- **Design Principle Applied:**
  - High Cohesion* → modules like User, Book, Exchange are internally consistent.
  - Reusability* → services can be extended (e.g., adding recommendation engine).
  - Separation of Concerns* → business logic is fully separated from UI and database handling.

# Data Layer

- Responsible for storing and retrieving persistent data.
- PostgreSQL (accessed via SQLAlchemy ORM)
- **Responsibilities:**

Stores all persistent data for Users, Books, and Exchanges.

Manages one-to-many and one-to-one relationships:

One User → many Books

One Book → one Exchange (in progress)

Performs CRUD operations (Create, Read, Update, Delete).

Ensures referential integrity between related tables.

- **Design Principle Applied:**

*Abstraction* → database queries hidden behind data-access functions.

*Data Integrity* → Enforced through foreign keys and object relationships.

Presentation layer

Mobile App(Flutter)



API Requests  
(HTTP)

Business Logic Layer

User Module  
Book Module  
Exchange Module  
Messages and  
Notifications



SQL Queries

Data Layer

Users Table  
Books Table  
Exchanges Table



# How it applies to our use case

## Example Flow: Requesting a Book

- **User (UI)** clicks “Request Book.”
- **Presentation Layer** sends a POST /exchange/request to backend API.
- **Business Logic Layer** validates the request: Checks if the book exists and is available -> Ensures both users are verified -> Creates a new record in Exchange.
- **Data Layer** stores the transaction details.
- **Business Logic Layer** triggers a notification.
- **UI Layer** displays confirmation and updates book status.

Each layer performs only its specific responsibility, preventing overlap and ensuring reliability.

# Design principles

| Principle      | Implementation Example                                                                           | Benefit                                                          |
|----------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| High Cohesion  | Book module only manages book data and actions.                                                  | Simplifies debugging and unit testing.                           |
| Loose Coupling | Flutter frontend interacts with backend via REST APIs only.                                      | Easier upgrades and maintenance, cleaner separation of concerns. |
| Reusability    | NotificationService and Exchange logic can be reused for future features like reminders or chat. | Reduces development effort for extensions.                       |
| Scalability    | Independent scaling of backend or database.                                                      | Handles more users/books efficiently.                            |
| Abstraction    | Data access hidden behind ORM models (SQLAlchemy).                                               | Prevents tight dependency on DB structure                        |



04

# Testing Strategy

Functional and Non-  
Functional Testing

# Functional Testing

| Module/Feature | What is Tested                          | Example Test Case                                                  |
|----------------|-----------------------------------------|--------------------------------------------------------------------|
| Authentication | Register, Login, Logout, Password rules | Test: Login with wrong password → Returns “Invalid credentials.”   |
| Books          | List, Search/Filter, View details       | Test: Fetch /books → Returns JSON with correct book-owner data.    |
| Trade/Exchange | Create, Accept, Decline, Status updates | Test: Send POST /exchange/request?name=User1 → Status = “Pending”. |
| Profile        | Edit profile, view owned books, history | Test: GET /profile returns name, email, list of books.             |
| Database       | Models, relationships, constraints      | Test: Deleting User removes their books (cascade delete).          |

# Non-Functional



- **Performance:** /books and /exchange API responses under 2s (p95) for 50 concurrent users
- **Security:** Backend enforces password hashing. No direct DB access from UI.
- **Reliability:** Consistent error messages from FastAPI (HTTPException handling).
- **Usability & Accessibility:** good and easy to use GUI. Flutter provides clear navigation
- **Compatibility:** Flutter ensures responsive performance on Android, iOS, Chrome (web build).

# When?

- **Day-to-day:**
  - Devs write **unit tests** with each feature
  - API **tests** added with each new endpoint
- **Mid-sprint:**
  - QA runs **feature tests** in **dev** environment
- **End of sprint:**
  - **Regression** suite (smoke + key paths)
  - **Performance spot-checks** on search & trade endpoints
  - **UAT** with product owner using a short checklist
- **Before Release:**
  - Full smoke test, verification of all endpoints, DB schema, and error handling.



# Who?

- **Developers:**
  - Unit tests
  - API tests
- **QA Lead:**
  - Writes test cases
  - Files bugs
- **Product Owner:**
  - Performs UAT against UML-defined user stories.
- **DevOps:**
  - Sets up test/staging env
  - Manages deployment pipeline.



# Project Management

Methodology: Agile Scrum



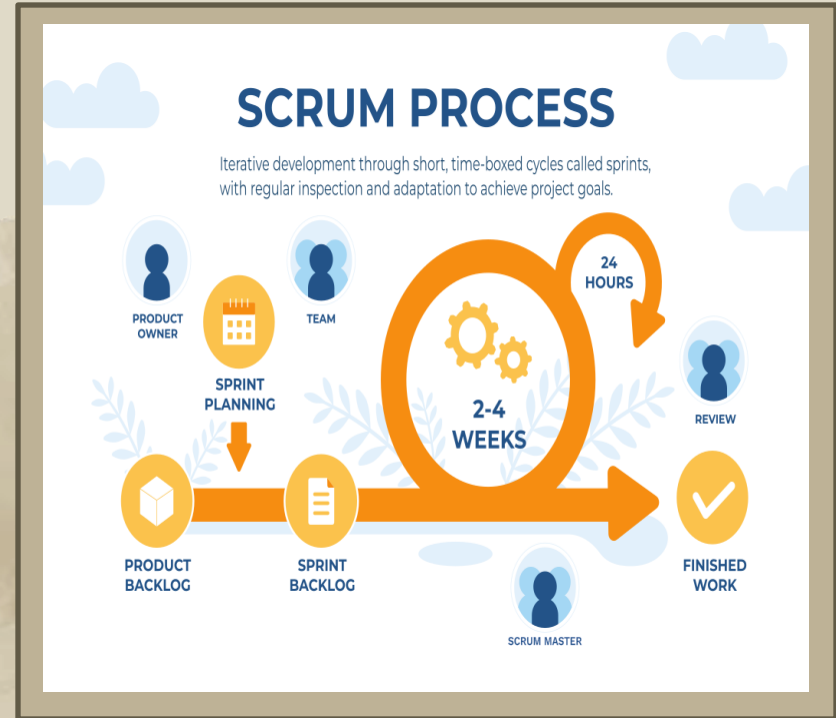


# Agile (Scrum)

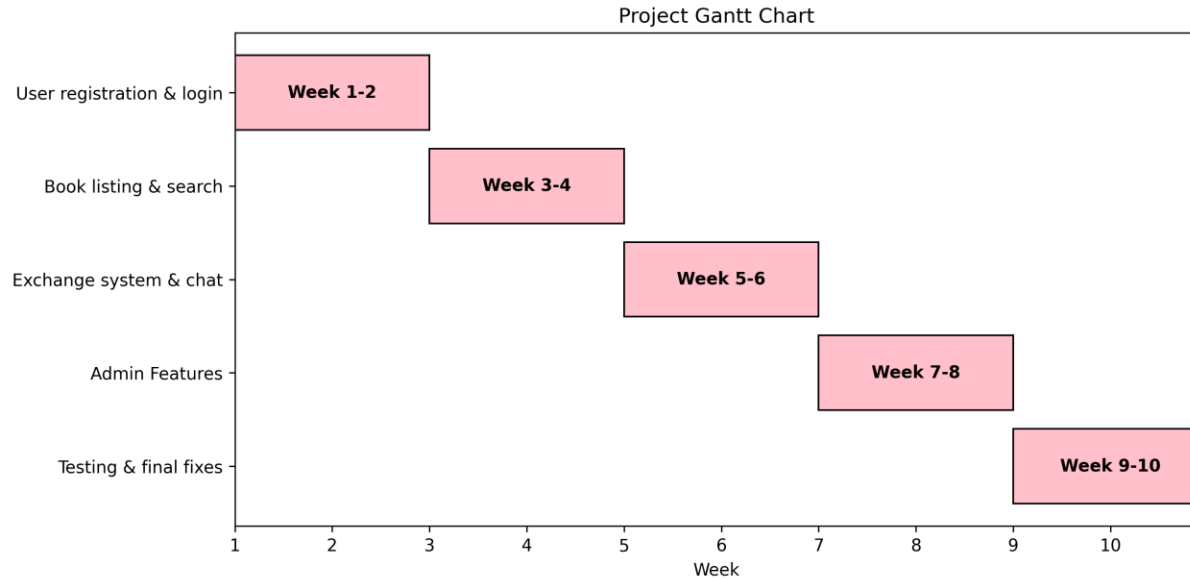
- Iterative, sprint-driven approach suitable for evolving user requirements.
- Development will be divided into 2 week sprints, allowing the team to focus on one set of features at a time.

## Benefits :

- Encourages iterative progress and continuous feedback.
- Easily accommodates changes or improvements during development



# Gantt Chart



- Each row represents a sprint, and the colored bar shows the duration of each phase.
- The chart helps visualize the flow of our project and how tasks are scheduled.

# Budget & Resource Overview

| Resource               | Details/Tools                                        | Cost            |
|------------------------|------------------------------------------------------|-----------------|
| Development Tools      | FastAPI, Flutter SDK, PostgreSQL (open-source)       | Free            |
| Hosting / Deployment   | Local / cloud testing (Heroku, Render, or localhost) | Low / Free tier |
| Design & Collaboration | Figma, GitHub, Teams                                 | Free plans      |
| Testing Tools          | Figma, GitHub, Trello / Jira                         | Free            |

# Team roles & Responsibilities :



Project Manager : Plan and track sprints, coordinate tasks, monitor deadlines, communicate updates



Developer(s) : Implement backend APIs, frontend interface, and integrate features



Tester : Perform functional and non-functional testing, track bugs, ensure quality



Designer : Create UI/UX wireframes, mockups, and maintain design consistency



Documentation Lead : Prepare UML diagrams, SDLC documentation, reports, and presentation slides



Each member has a clearly defined role, which improves accountability and productivity

# Conclusion



“ReBook promotes sustainability and reading culture through a user-centered, scalable, and maintainable book exchange platform — designed using Agile methodology, modeled with UML, tested for reliability, and managed using modern project management principles.”



# Thanks

Do you have any questions?

CREDITS: This presentation template was  
created by **Slidesgo**, including icons by  
**Flaticon**, infographics & images by **Freepik**

